Федеральное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и компьютерной техники

Дисциплина: *«Веб-программирование»*

Отчёт
По лабораторной работе №3
Вариант **14**

Выполнил:
Крамарь Михаил Романович
P3213

Проверил:
Бойко Владислав Алексеевич

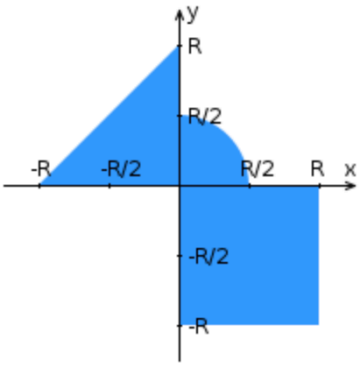
г. Санкт-Петербург
2026

Содержание

Текст задания 3
Ход работы 5
Ссылки на полезные ресурсы.....7
Вывод.....8

Текст задания

Разработать веб-приложение на базе сервлетов и JSP, определяющее попадание точки на координатной плоскости в заданную область.



изменение X: Text (-5 ... 3)

изменение Y: Text (-5 ... 5)

изменение R: Radio {'1','1.5','2','2.5','3'}

метод HTTP: GET

список результатов сохранять в HTTP-сессии

Приложение должно быть реализовано в соответствии с шаблоном MVC и состоять из следующих элементов:

- **ControllerServlet**, определяющий тип запроса и, в зависимости от того, содержит ли запрос информацию о координатах точки и радиусе, делегирующий его обработку одному из перечисленных ниже компонентов. Все запросы внутри приложения должны передаваться этому сервлету (по методу GET в зависимости от варианта задания), остальные сервлеты с веб-страниц напрямую вызываться не должны.
- **AreaCheckServlet**, осуществляющий проверку попадания точки в область на координатной плоскости и формирующий HTML-страницу с результатами проверки. Должен обрабатывать все запросы, содержащие сведения о координатах точки и радиусе области.
- **Страница JSP**, формирующая HTML-страницу с веб-формой. Должна обрабатывать все запросы, не содержащие сведений о координатах точки и радиусе области.

Разработанная страница JSP должна содержать:

- «Шапку», содержащую ФИО студента, номер группы и номер варианта.
- Форму, отправляющую данные на сервер.
- Набор полей для задания координат точки и радиуса области в соответствии с вариантом задания: Text (-5 ... 3) для координаты по оси X, Text (-5 ... 5) для координаты по оси Y, Radio {'1','1.5','2','2.5','3'} для задания радиуса области.
- Сценарий на языке JavaScript, осуществляющий валидацию значений, вводимых пользователем в поля формы.

- Интерактивный элемент, содержащий изображение области на координатной плоскости (в соответствии с вариантом задания) и реализующий следующую функциональность: если радиус области установлен, клик курсором мыши по изображению должен обрабатываться JavaScript-функцией, определяющей координаты точки и отправляющей их на сервер для проверки; в противном случае после клика должно выводиться сообщение о невозможности определения координат. После проверки изображение должно обновляться с учётом результатов.
- Таблицу с результатами предыдущих проверок. Список результатов должен браться из HTTP-сессии.

Страница, возвращаемая AreaCheckServlet, должна содержать:

- Таблицу, содержащую полученные параметры.
- Результат вычислений — факт попадания или непопадания точки в область.
- Ссылку на страницу с веб-формой для формирования нового запроса.

Разработанное веб-приложение необходимо развернуть на сервере WildFly. Сервер должен быть запущен в standalone-конфигурации, порты должны быть настроены в соответствии с выданным portbase, доступ к http listener'у должен быть открыт для всех IP.

Ход работы

1. Изучение предметной области и документации

Сначала я изучил текст задания и методические указания к лабораторной работе, чтобы разобраться в принципах работы сервлетов, JSP и взаимодействия между клиентской и серверной частями веб-приложения.

Я изучил жизненный цикл сервлетов, особенности обработки HTTP-запросов, работу контейнера сервлетов. Также разобрался с архитектурным шаблоном MVC и его реализацией в Java EE.

2. Настройка окружения и веб-сервера

Далее я настроил окружение для разработки и запуска приложения. В качестве сервера приложений использовал WildFly, который предоставляет контейнер сервлетов и среду для выполнения веб-приложения.

Я настроил структуру проекта, зависимости и конфигурацию развёртывания. Также разобрался с назначением дескриптора развёртывания web.xml, настройкой сервлетов и их URL-mapping. После этого проверил локальный запуск приложения на сервере WildFly.

3. Разработка серверной части на Java

Затем я реализовал серверную часть приложения с использованием сервлетов. Основным компонентом стал ControllerServer, который принимает HTTP-запросы и определяет, какой обработчик должен их выполнить.

Для проверки попадания точки в заданную область я реализовал отдельный AreaCheckServlet. Он получает координаты и радиус, преобразует переданные параметры в числовой формат, проверяет их корректность и выполняет вычисление попадания. Если данные некорректны, сервер возвращает соответствующий HTTP-статус.

Результат проверки хранится в отдельном объекте ResultRecord, который я взял из прошлой лабораторной работы. Для работы с историей результатов я создал класс RecordHistory, который сохраняет записи в HTTP-сессии пользователя. Таким образом, история проверок сохраняется между запросами, но при этом у каждого пользователя она своя.

4. Разработка клиентской части и JSP

После этого я переработал клиентскую часть приложения, используя JSP в качестве представления. Основная страница содержит форму для ввода координат и радиуса, таблицу результатов и Canvas с изображением проверяемой области.

Для взаимодействия с сервером я сохранил два способа отправки запросов. При отправке формы пользователь получает полноценную HTML-страницу с результатом проверки, а при нажатии на Canvas запрос выполняется асинхронно через Fetch API (это сделано,

чтобы было видно как нажимается точка, просто было бы не очень удобно, тыкая на Canvas, попадать постоянно на другую страницу, не успевая увидеть куда тыкнул). В последнем случае сервер возвращает JSON, после чего таблица результатов обновляется без полной перезагрузки страницы.

Также я реализовал отображение истории проверок, получаемой из HTTP-сессии, и возможность её очистки. Для обработки запросов использовал механизм перенаправления управления внутри приложения через RequestDispatcher, благодаря чему все динамические запросы проходят через контроллер.

5. Возникшие проблемы и их решение

В процессе разработки возникли проблемы с обработкой некорректных параметров и маршрутизацией запросов. В частности, потребовалось разделить ошибки некорректного ввода и обращения к недоступным ресурсам. Для этого я добавил серверную проверку координат и радиуса, а также настроил обработку HTTP-ошибок через web.xml.

Отдельное внимание пришлось уделить сохранению истории результатов. Изначально было важно правильно организовать хранение данных между запросами, поэтому я использовал HttpSession, а не общий список для всех пользователей. Это позволило изолировать историю каждого пользователя.

6. Развёртывание приложения

В завершение я подготовил приложение к развёртыванию, собрал его в WAR-архив и перенёс на сервер Helios. Для запуска использовал WildFly, настроив необходимые параметры сервера и пути к рабочим директориям.

Ссылки на полезные ресурсы

Исходный код:

- https://github.com/moon2light-gif/ITMIUMIU/tree/main/3-4%E2%80%93WebProgramming/Lab3

Веб-сайт:

У Гелиоса все кастомные порты, к сожалению, закрытые

- Использованный мной ПОРТ := *50251*
- ssh -L <ПОРТ>:localhost:<ПОРТ> s502511@helios.cs.ifmo.ru -p 2222
- *http://localhost:<ПОРТ>/lab3/*

Вывод

В ходе выполнения лабораторной работы №3 мной были усовершенствованы навыки работы с веб-серверами и создания веб-приложений, я узнал, что такое WildFly и познакомился с Java/Jakarta EE. Я подробно изучил Servlet API и JSP, реализовал шаблон MVC и разобрался с Web Audio API в качестве дополнительного задания.